

Domain Name System (DNS) & E-Mail

CEN 223 - Internet Communication

Assoc. Prof. Dr. Fatih ABUT
Department of Computer Engineering
Çukurova University



Domain Name System (DNS) was proposed
in 1983 by Paul Mockapetris

Host Names vs. IP addresses

- ▶ Host names
 - ▶ Mnemonic name appreciated by humans
 - ▶ Variable length, alpha-numeric characters
 - ▶ Provide little (if any) information about location
 - ▶ Examples: www.cnn.com and ftp.eurocom.fr
- ▶ IP addresses
 - ▶ Numerical address appreciated by routers
 - ▶ Fixed length, binary number
 - ▶ Hierarchical, related to host location
 - ▶ Examples: 64.236.16.20 and 193.30.227.161

“What's the IP address of mail.cis.udel.edu?”

“128.4.40.12”

“What's the name of the host at 128.4.40.12?”

“mail.cis.udel.edu”

Separating Naming and Addressing

- ▶ Names are easier to remember
 - ▶ `www.cnn.com` vs. `64.236.16.20`
- ▶ Addresses can change underneath
 - ▶ Move `www.cnn.com` to `173.15.201.39`
 - ▶ E.g., renumbering when changing providers
- ▶ Name could map to multiple IP addresses
 - ▶ `www.cnn.com` to multiple replicas of the Web site
- ▶ Map to different addresses in different places
 - ▶ Address of a nearby copy of the Web site
 - ▶ E.g., to reduce latency, or return different content
- ▶ Multiple names for the same address
 - ▶ E.g., aliases like `ee.mit.edu` and `cs.mit.edu`

Strawman Solution: Central Server

- ▶ Central server
 - ▶ One place where all mappings are stored
 - ▶ All queries go to the central server
- ▶ Many practical problems
 - ▶ Single point of failure
 - ▶ High traffic volume
 - ▶ Distant centralized database
 - ▶ Single point of update
 - ▶ Does not scale

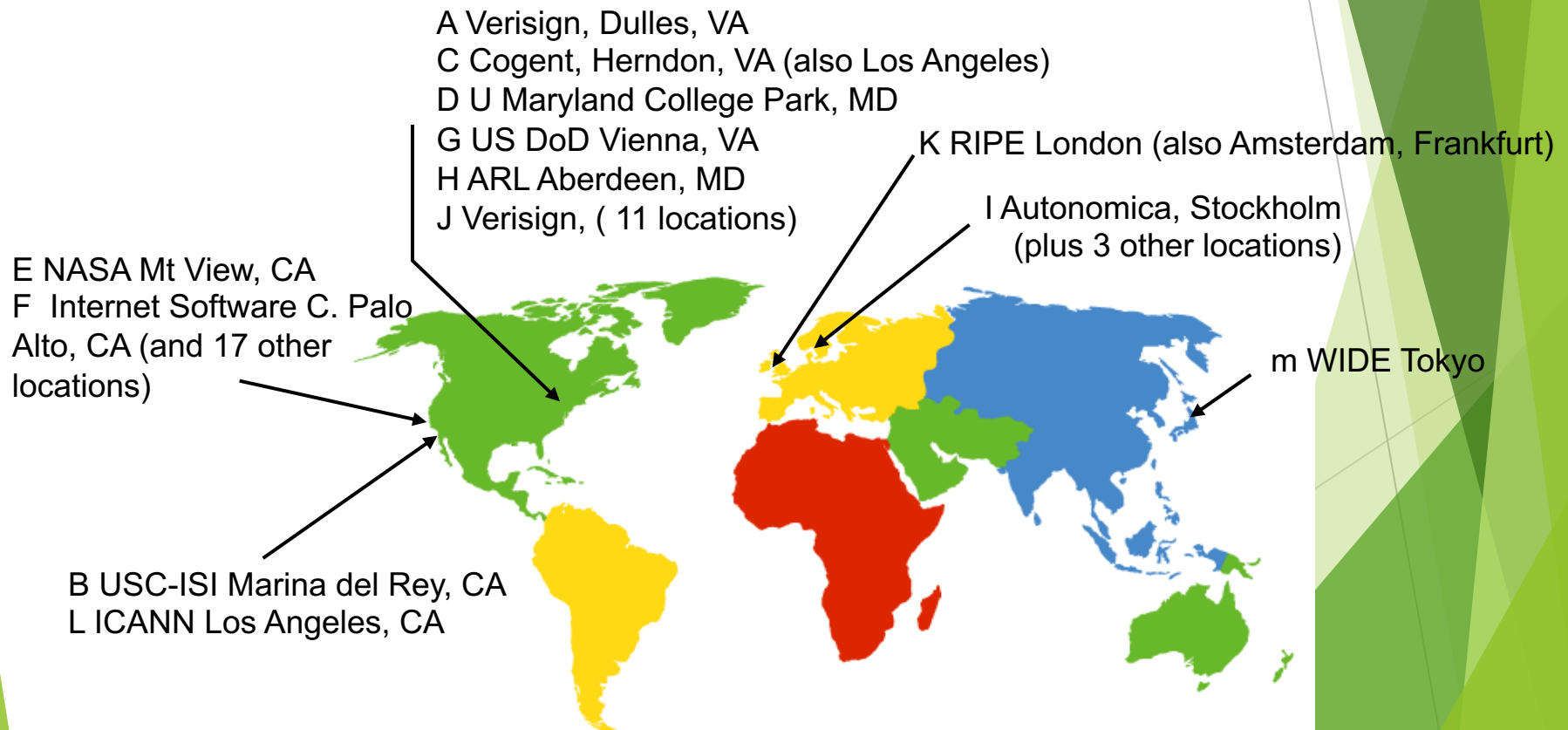
Need a distributed, hierarchical collection of servers

Domain Name System (DNS)

- ▶ Properties of DNS
 - ▶ Hierarchical name space divided into zones
 - ▶ Distributed over a collection of DNS servers
- ▶ Hierarchy of DNS servers
 - ▶ Root servers
 - ▶ Top-level domain (TLD) servers
 - ▶ Authoritative DNS servers
- ▶ Performing the translations
 - ▶ Local DNS servers
 - ▶ Resolver software

DNS Root Servers

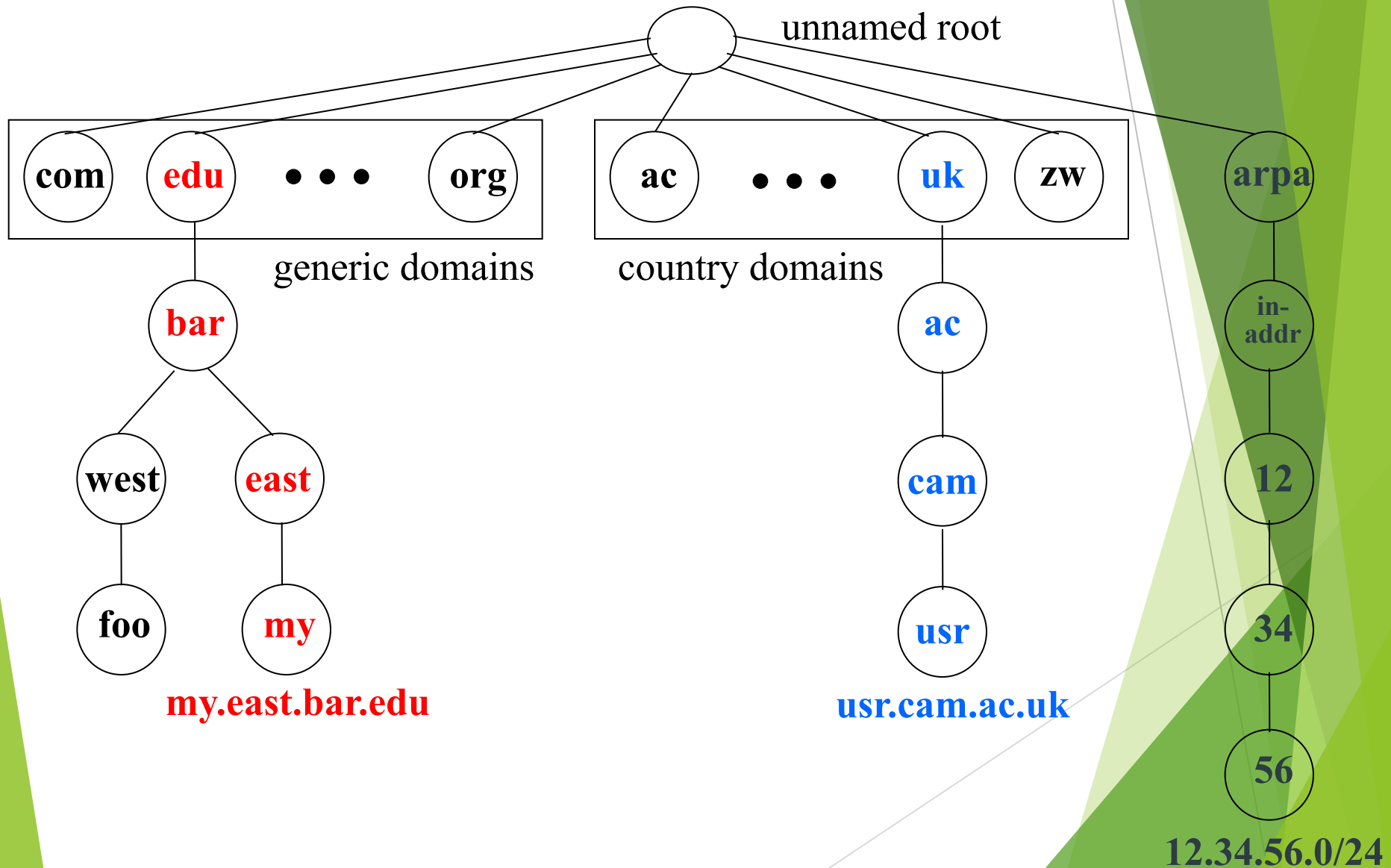
- ▶ 13 root servers (see <http://www.root-servers.org/>)
- ▶ Labeled A through M



TLD and Authoritative DNS Servers

- ▶ Top-level domain (TLD) servers
 - ▶ Generic domains (e.g., com, org, edu)
 - ▶ Country domains (e.g., uk, fr, ca, jp)
 - ▶ Typically managed professionally
 - ▶ Network Solutions maintains servers for “com”
 - ▶ Educause maintains servers for “edu”
- ▶ Authoritative DNS servers
 - ▶ Provide public records for hosts at an organization
 - ▶ For the organization’s servers (e.g., Web and mail)
 - ▶ Can be maintained locally or by a service provider

Distributed Hierarchical Database

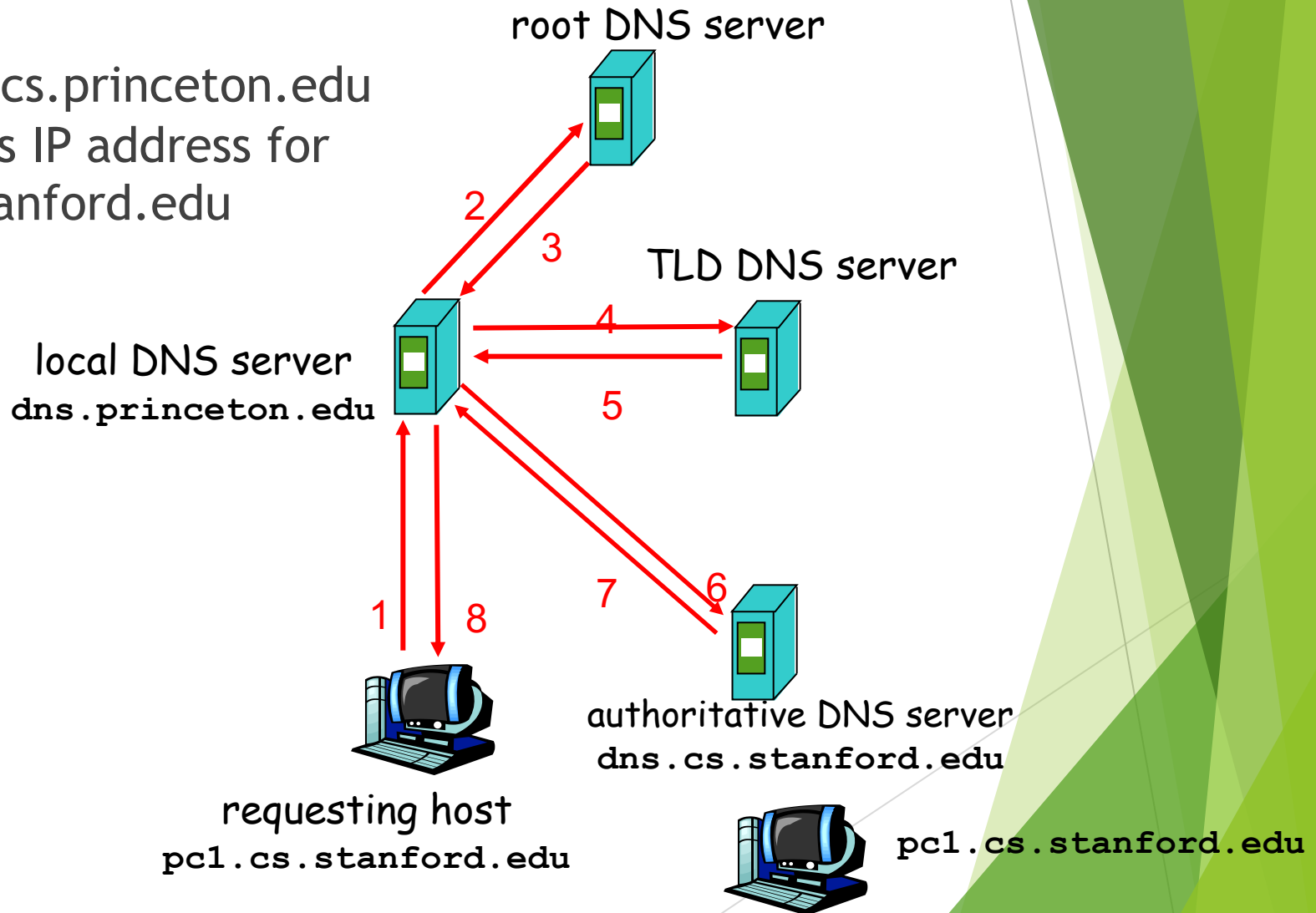


Using DNS

- ▶ Local DNS server (“default name server”)
 - ▶ Usually near the end hosts who use it
 - ▶ Local hosts configured with local server (e.g., `/etc/resolv.conf`) or learn the server via DHCP
- ▶ Client application
 - ▶ Extract server name (e.g., from the URL)
 - ▶ Do *gethostbyname()* to trigger resolver code
- ▶ Server application
 - ▶ Extract client IP address from socket
 - ▶ Optional *gethostbyaddr()* to translate into name

Example

Host at cs.princeton.edu
wants IP address for
cs.stanford.edu



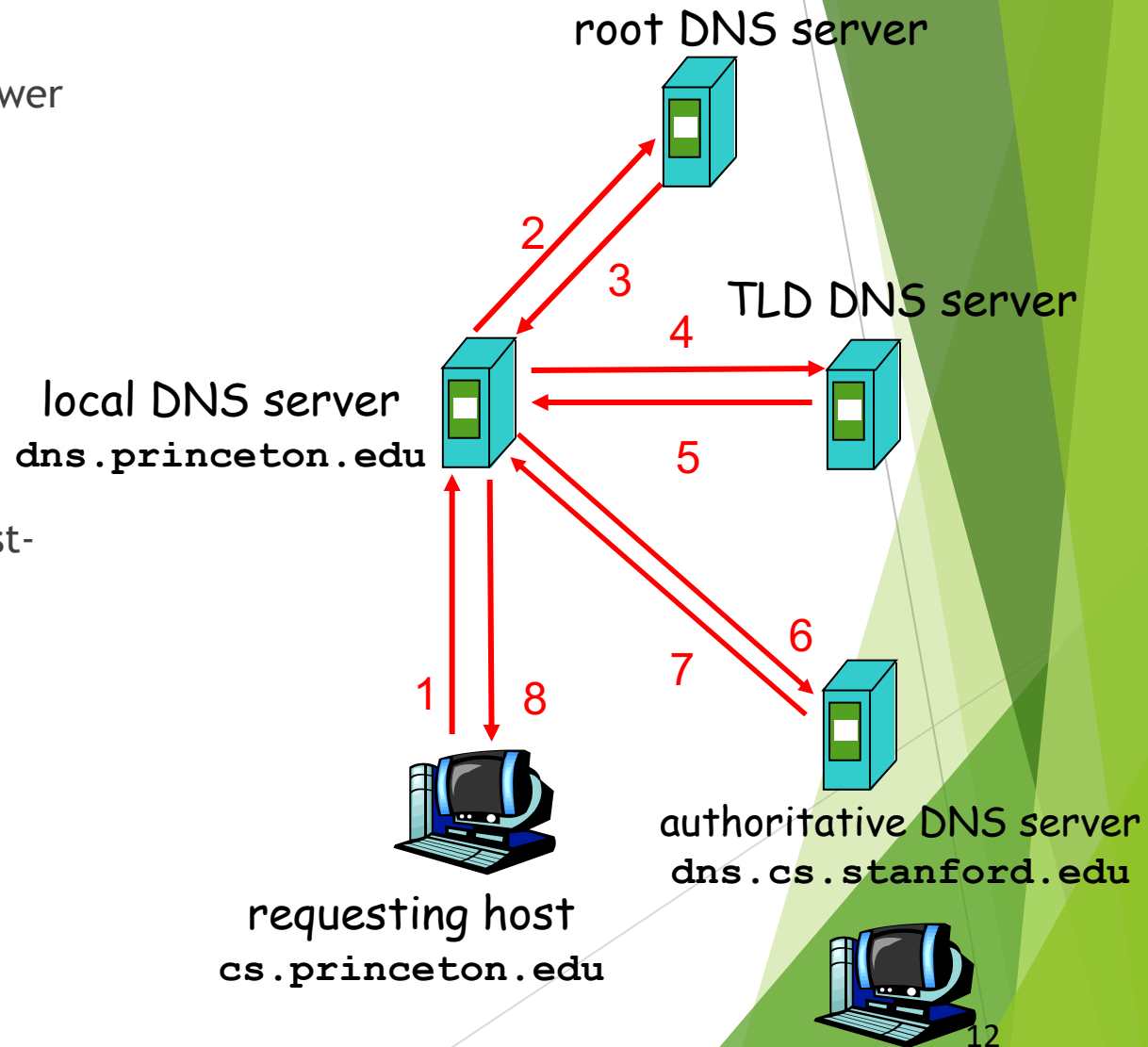
Recursive vs. Iterative Queries

► Recursive query

- Ask server to get answer for you
- E.g., request 1 and response 8

► Iterative query

- Ask server who to ask next
- E.g., all other request-response pairs



DNS Caching

- ▶ Performing all these queries take time
 - ▶ And all this before the actual communication takes place
 - ▶ E.g., 1-second latency before starting Web download
- ▶ Caching can substantially reduce overhead
 - ▶ The top-level servers very rarely change
 - ▶ Popular sites (e.g., www.cnn.com) visited often
 - ▶ Local DNS server often has the information cached
- ▶ How DNS caching works
 - ▶ DNS servers cache responses to queries
 - ▶ Responses include a “time to live” (TTL) field
 - ▶ Server deletes the cached entry after TTL expires

Negative Caching

- ▶ Remember things that don't work
 - ▶ Misspellings like www.cnn.comm and www.cnnn.com
 - ▶ These can take a long time to fail the first time
 - ▶ Good to remember that they don't work
 - ▶ ... so the failure takes less time the next time around

DNS Resource Records

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

- Type=A

- name is hostname
- value is IP address

- ▶ Type=NS

- ▶ name is domain (e.g. foo.com)
- ▶ value is hostname of authoritative name server for this domain

- Type=CNAME

- name is alias name for some “canonical” (the real) name
www.ibm.com is really
servereast.backup2.ibm.com
- value is canonical name

- Type=MX

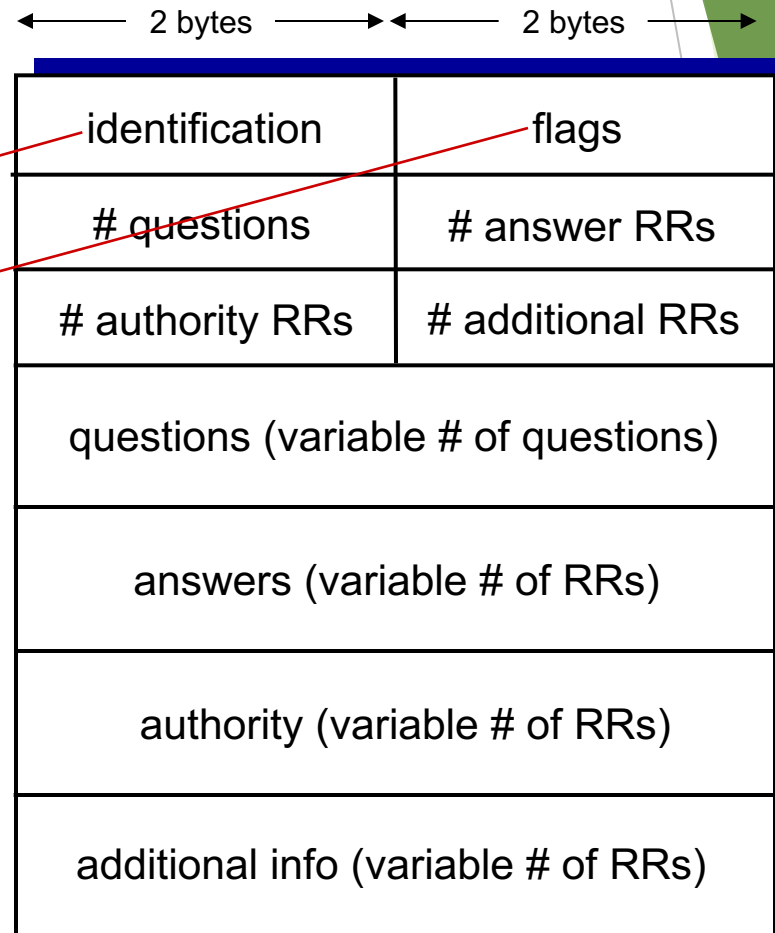
- value is name of mailserver associated with name

DNS protocol, messages

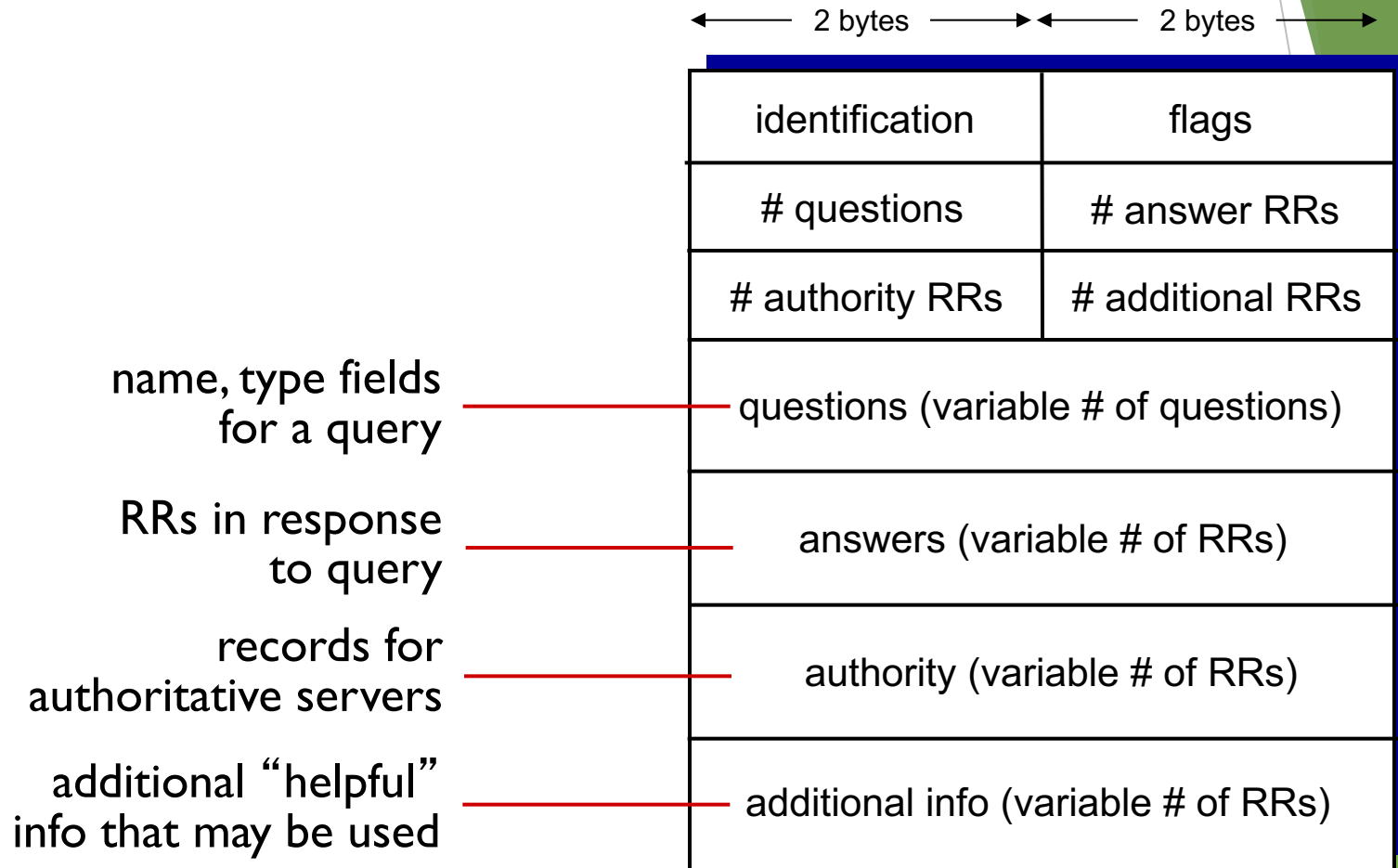
- *query* and *reply* messages, both with same *message format*

msg header

- ❖ **identification**: 16 bit # for query, reply to query uses same #
- ❖ **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol, messages

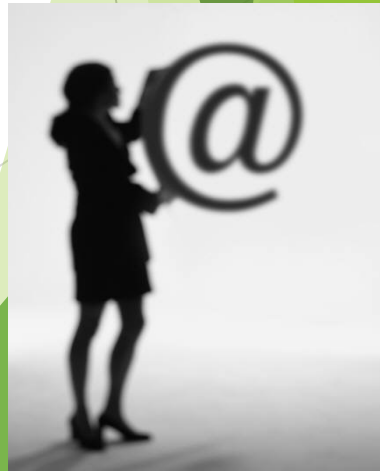


Reliability

- ▶ DNS servers are replicated
 - ▶ Name service available if at least one replica is up
 - ▶ Queries can be load balanced between replicas
- ▶ UDP used for queries
 - ▶ Need reliability: must implement this on top of UDP
- ▶ Try alternate servers on timeout
 - ▶ Exponential backoff when retrying same server
- ▶ Same identifier for all queries
 - ▶ Don't care which server responds

Inserting Resource Records into DNS

- ▶ Example: just created startup “FooBar”
- ▶ Register foobar.com at Network Solutions
 - ▶ Provide registrar with names and IP addresses of your authoritative name server (primary and secondary)
 - ▶ Registrar inserts two RRs into the com TLD server:
 - ▶ (foobar.com/, dns1.foobar.com, NS)
 - ▶ (dns1.foobar.com, 212.212.212.1, A)
- ▶ Put in authoritative server dns1.foobar.com
 - ▶ Type A record for www.foobar.com
 - ▶ Type MX record for foobar.com

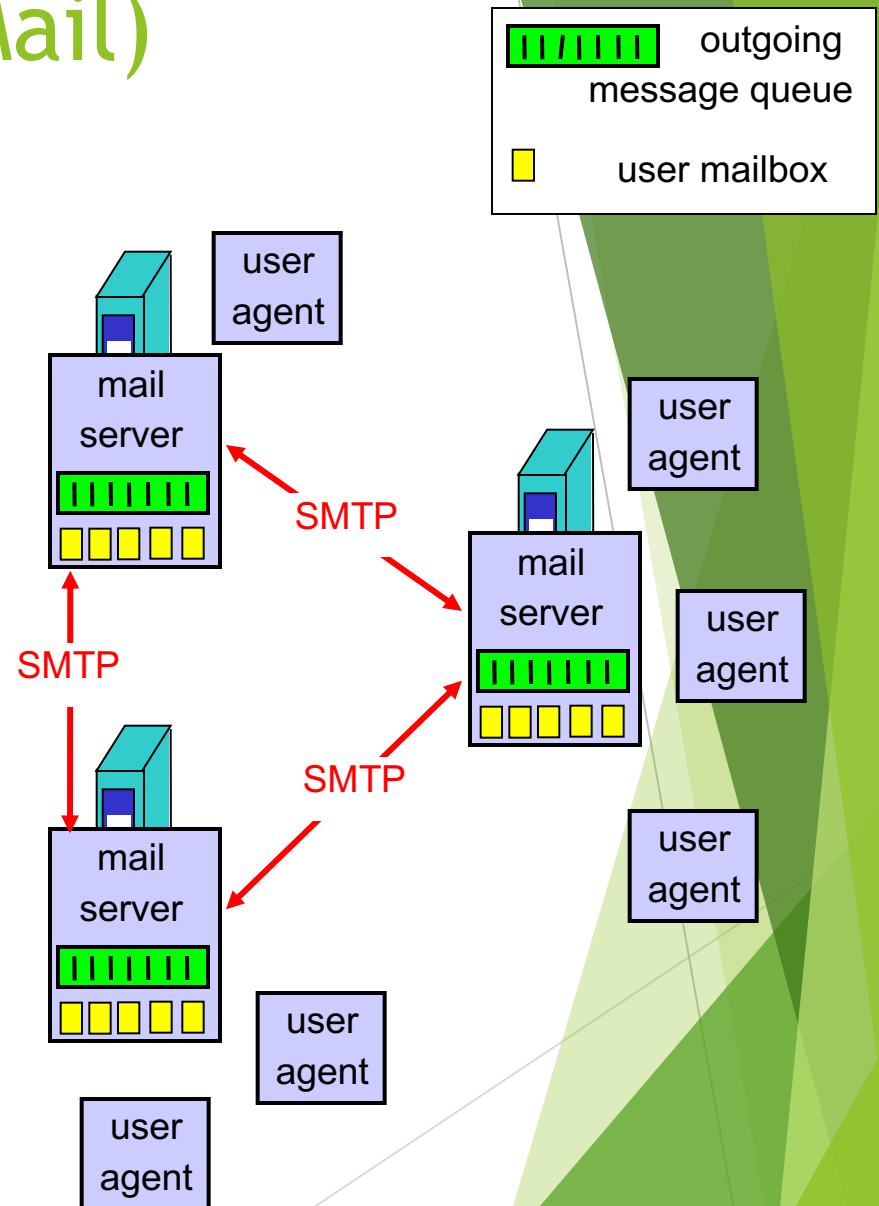


Electronic Mail (e-Mail)

- ▶ **Three major components:**
 - ▶ user agents
 - ▶ mail servers
 - ▶ simple mail transfer protocol: SMTP

User Agent

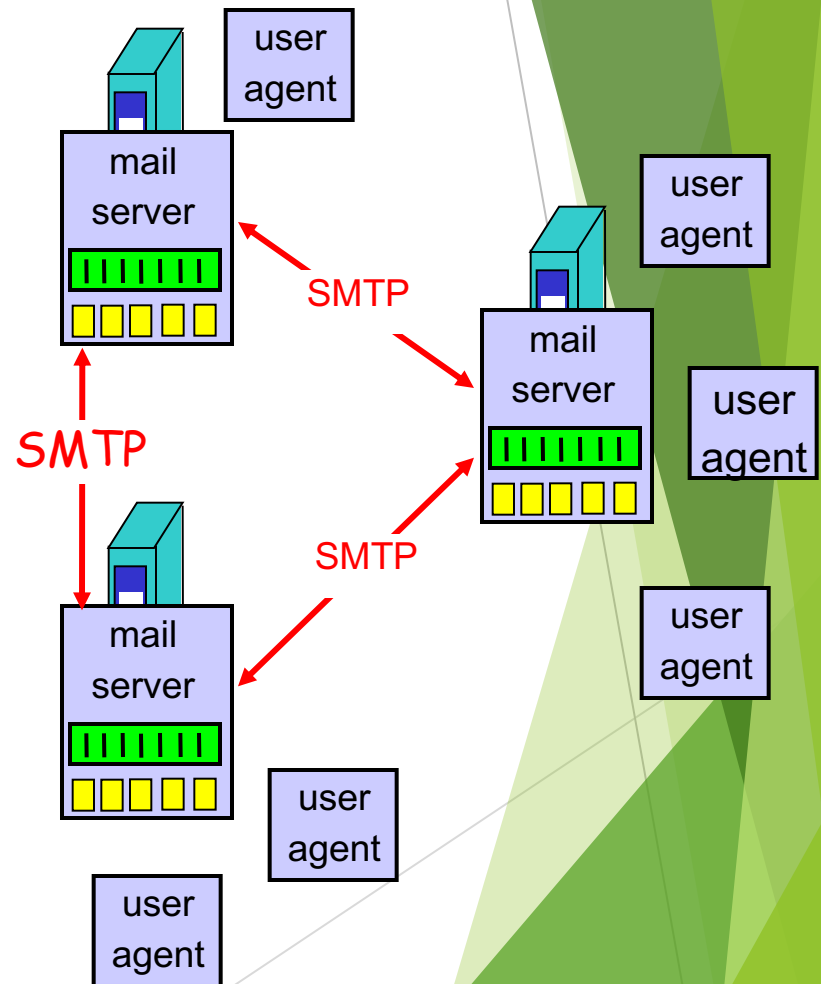
- ▶ a.k.a. “mail reader”
- ▶ composing, editing, reading mail messages
- ▶ e.g., Outlook, Mozilla Thunderbird, Apple Mail
- ▶ outgoing, incoming messages stored on server



Electronic Mail: mail servers

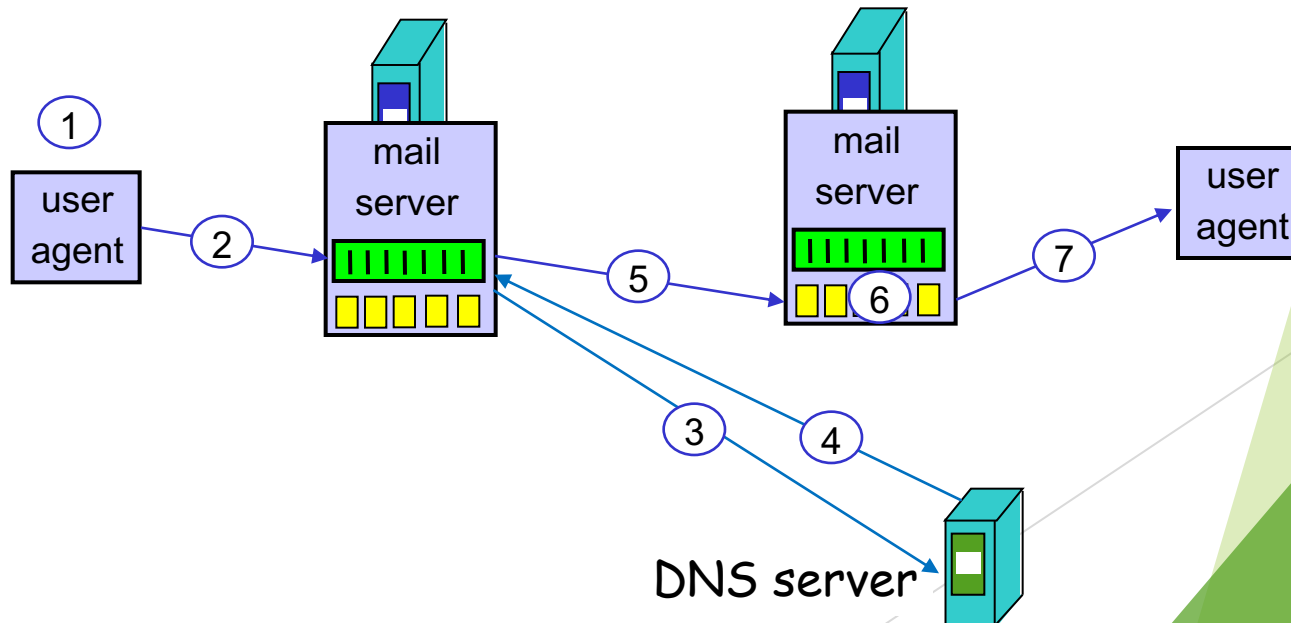
► Mail Servers

- **mailbox** contains incoming messages for user
- **message queue** of outgoing (to be sent) mail messages
- **SMTP protocol** between mail servers to send email messages
 - ❖ client: sending mail server
 - ❖ “server”: receiving mail server



Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message and "to" bob@some school.edu
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) The part before the @ sign is the local part of the address, often the username of the recipient, and the part after the @ sign is a domain name. The mail server looks up this domain name in the Domain Name System to find the mail exchange servers accepting messages for that domain (i.e., some school.edu).
- 4) The DNS server responds with an MX record listing the mail exchange servers for that domain.
- 5) Client side of SMTP opens TCP connection with Bob's mail server using the resolved IP address and sends Alice's message over the TCP connection
- 6) Bob's mail server places the message in Bob's mailbox
- 7) Bob invokes his user agent to read message



Electronic Mail: SMTP [RFC 2821]

- ▶ uses TCP to reliably transfer email message from client to server, port 25
- ▶ direct transfer: sending server to receiving server
- ▶ three phases of transfer
 - ❖ handshaking (greeting)
 - ❖ transfer of messages
 - ❖ closure
- ▶ command/response interaction
 - ❖ **commands**: ASCII text
 - ❖ **response**: status code and phrase
- ▶ messages must be in 7-bit ASCII

Try SMTP interaction for yourself:

- ▶ `telnet servername 25`
- ▶ see 220 reply from server
- ▶ enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

Trace it - does your mail data go in the clear?

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Mail message format

- ▶ SMTP: protocol for exchanging email msgs

- ▶ RFC 822: standard for text message format:

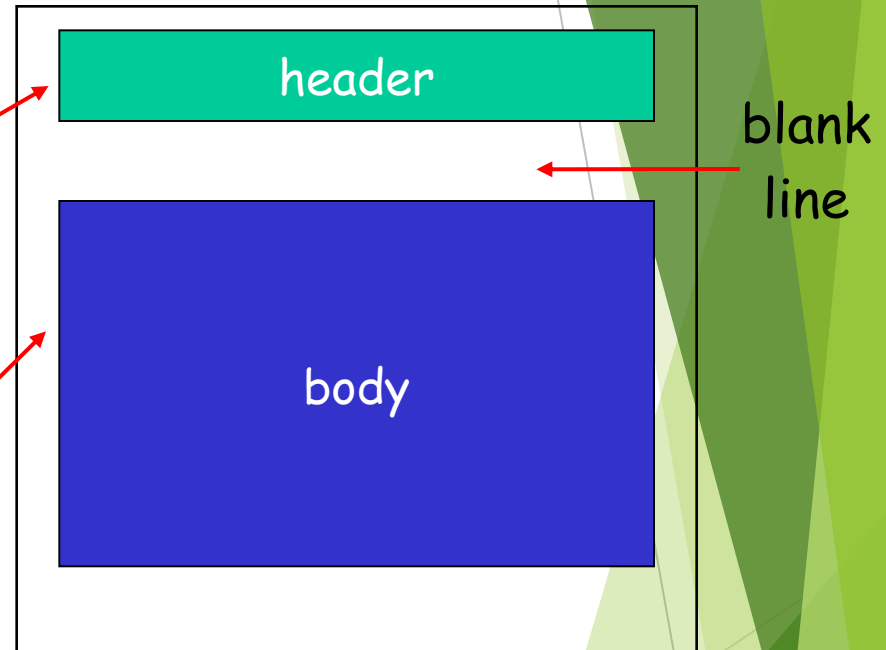
? header lines, e.g.,

- ❖ To:
- ❖ From:
- ❖ CC:
- ❖ Subject:
- ❖ Date:

different from SMTP commands!

? body

- ❖ the “message”, ASCII characters only



Message format: multimedia extensions

- ▶ MIME: multimedia mail extension, RFC 2045, 2056
- ▶ additional lines in msg header declare MIME content type

MIME version

method used
to encode data

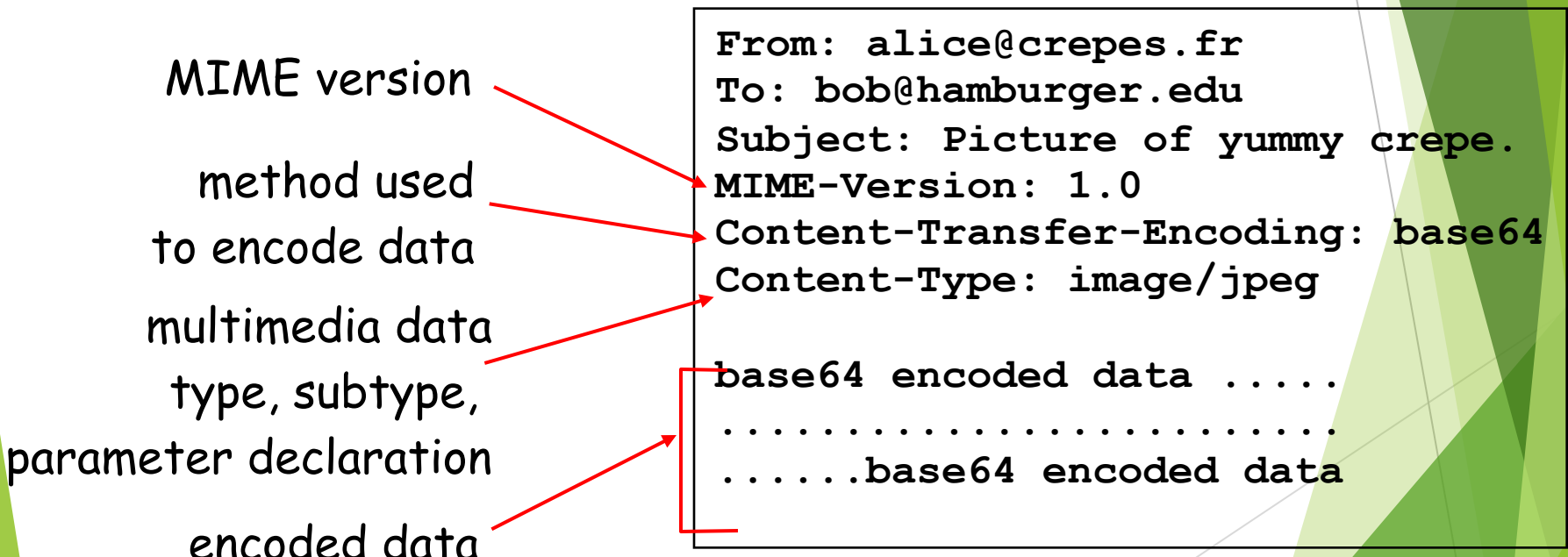
multimedia data
type, subtype,

parameter declaration

encoded data

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.....base64 encoded data
```

A diagram illustrating the components of a MIME message. On the left, five labels are listed: 'MIME version', 'method used to encode data', 'multimedia data type, subtype,', 'parameter declaration', and 'encoded data'. Red arrows point from these labels to the corresponding parts of a sample MIME message. The message is enclosed in a black rectangular box. The arrows point to 'MIME-Version: 1.0', 'Content-Transfer-Encoding: base64', 'Content-Type: image/jpeg', the first line of the body 'base64 encoded data', and the second line of the body '.....base64 encoded data'.

MIME types: Extensible

Content-Type: type/subtype; parameters

Text

- ▶ example subtypes:
`plain, html`

Image

- ▶ example subtypes:
`jpeg, gif`

Audio

- ▶ example subtypes:
`basic` (8-bit mu-law encoded), `32kadpcm` (32 kbps coding)

Video

- ▶ example subtypes:
`mpeg, quicktime`

Application

- ▶ other data that must be processed by reader before “viewable”
- ▶ example subtypes:
`msword, octet-stream`

Multipart Type

From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary=98766789

--98766789

Content-Transfer-Encoding: quoted-printable
Content-Type: text/plain

Dear Bob,
Please find a picture of a crepe.

--98766789

Content-Transfer-Encoding: base64
Content-Type: image/jpeg

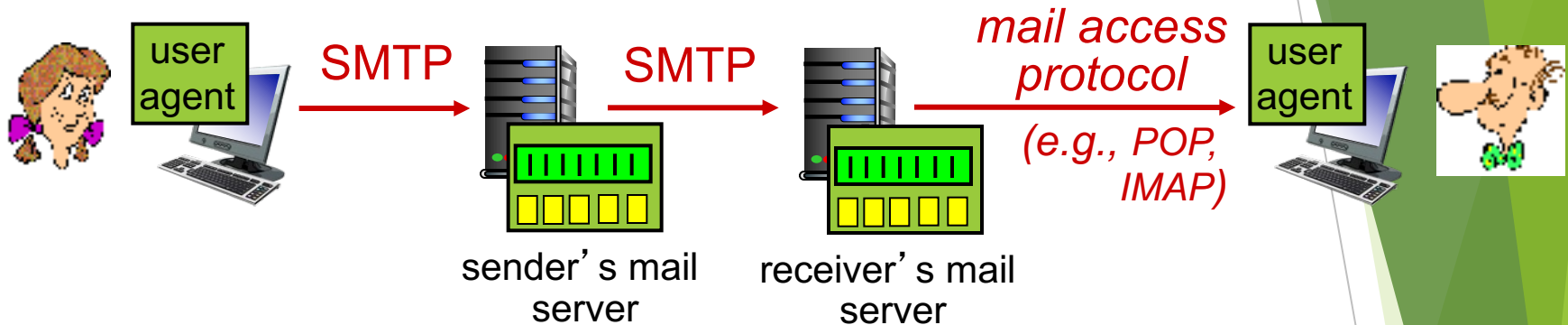
base64 encoded data

.....

.....base64 encoded data

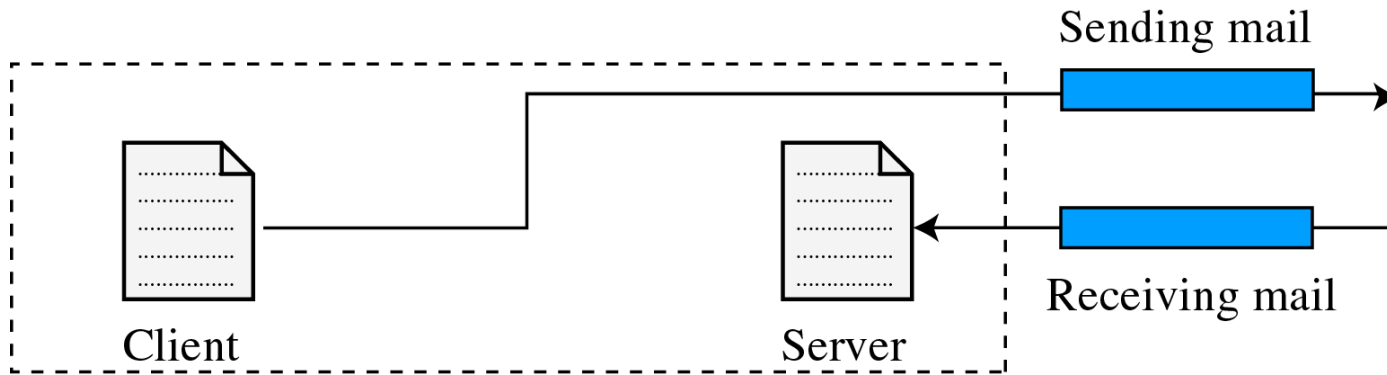
--98766789--

Mail access protocols

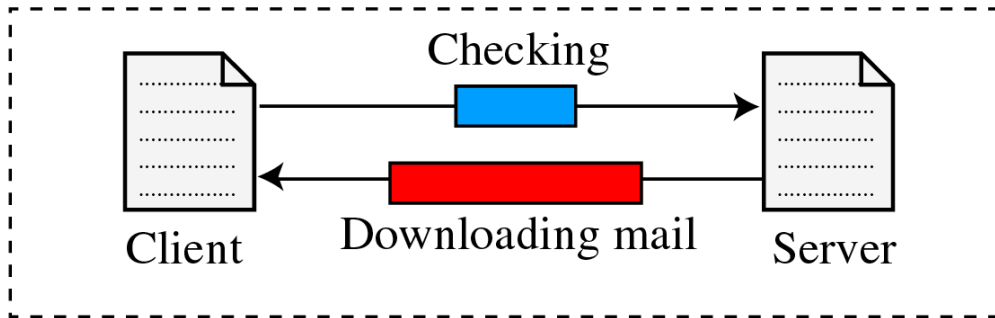


- ▶ **SMTP:** delivery/storage to receiver's server
- ▶ Mail access protocol: retrieval from server
 - ▶ **POP:** Post Office Protocol [RFC 1939]: authorization, download
 - ▶ **IMAP:** Internet Mail Access Protocol [RFC 1730]: more features, including manipulation of stored msgs on server
 - ▶ **HTTP:** Gmail, Hotmail, Yahoo! Mail, etc.

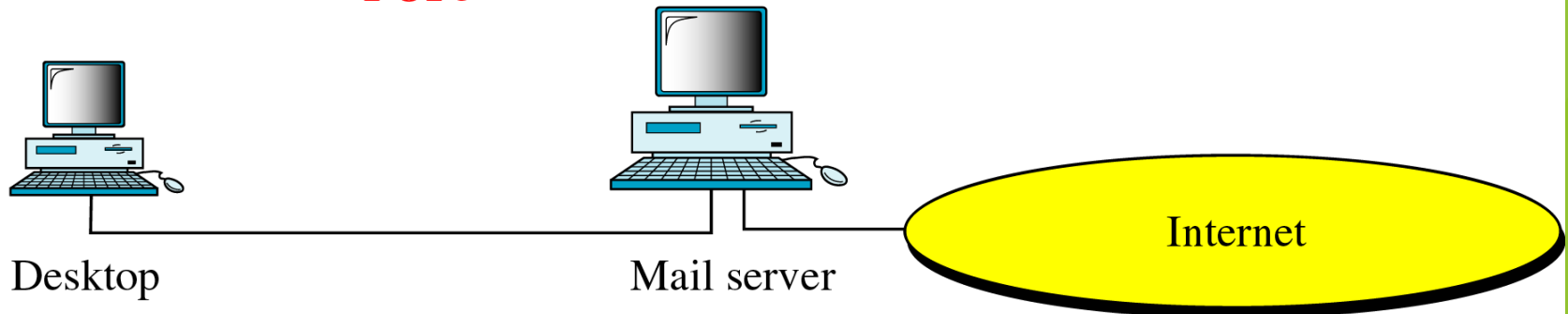
POP3 and SMTP



SMTP



POP3



POP3 protocol

► authorization phase

- ❑ client commands:
 - ❖ **user**: declare username
 - ❖ **pass**: password
- ❑ server responses
 - ❖ **+OK**
 - ❖ **-ERR**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

transaction phase, client:

- ❑ **list**: list message numbers
- ❑ **retr**: retrieve message by number
- ❑ **dele**: delete
- ❑ **quit**

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```


IMAP - Commands

- ▶ login
- ▶ list
- ▶ status
- ▶ examine
- ▶ select
- ▶ create, delete, rename
- ▶ fetch
- ▶ store
- ▶ close
- ▶ expunge
- ▶ copy
- ▶ idle
- ▶ lsub, subscribe, unsubscribe
- ▶ logout
- ▶ capability, getquotaroot, getacl

POP3 (more) and IMAP

► More about POP3

- Previous example uses “download and delete” mode.
- Bob cannot re-read e-mail if he changes client
- “Download-and-keep”: copies of messages on different clients
- POP3 is stateless across sessions

► IMAP

- Keep all messages in one place: the server
- Allows user to organize messages in folders
- IMAP keeps user state across sessions:
 - names of folders and mappings between message IDs and folder name

SMTP vs. HTTP

- ▶ SMTP uses persistent connections
- ▶ SMTP requires message (header & body) to be in 7-bit ASCII

comparison with HTTP:

- ▶ HTTP: pull
- ▶ SMTP: push
- ▶ both have ASCII command/response interaction, status codes
- ▶ HTTP: each object encapsulated in its own response msg
- ▶ SMTP: multiple objects sent in multipart msg